

CSC1018 - Logic

Reasoning in Propositional Logic

David Sinclair

CSC1018 David Sinclair ©2024

Decision Procedures

Let $\mathcal{U} \subseteq \mathcal{F}$ be a set of formulae. Then a *decision procedure for $\mathcal{U}$* is an algorithm that determines if an arbitrary formula $A \in \mathcal{F}$ is contained in $\mathcal{U}$.

If $\mathcal{U}$ is the set of satisfiable formulae, then the decision procedure for $\mathcal{U}$ is a *decision procedure for satisfiability*. If $\mathcal{U}$ is the set of valid formulae, then the decision procedure for $\mathcal{U}$ is a *decision procedure for validity*.

Alternatively, if we have a decision procedure for satisfiability, then we can determine A is valid if we can determine $\neg A$ is not satisfiable. This type of decision procedure is called a *refutation procedure*.

Logical Consequence

If $\mathcal{U}$ is a set of formulae and A is a formula, then A is a *logical consequence* of $\mathcal{U}$, denoted $\mathcal{U} \models A$, iff every model of $\mathcal{U}$ is a model of A .

A does not need to be true in every possible interpretation, only those interpretation that satisfy $\mathcal{U}$.

Example:

If $A = (p \wedge r) \vee (\neg q \wedge \neg r)$, then A is a logical consequence of $\{p, \neg q\}$, denoted $\{p, \neg q\} \models A$ since when $\mathcal{I}(p) = \text{true}$ and $\mathcal{I}(q) = \text{false}$, A is true.

Semantic Tableaux

A *literal* is an atom or its negation.

For any atom p , $\{p, \neg p\}$ is a *complementary pair of literals*.

For any formula A , $\{A, \neg A\}$ is a *complementary pair of formulae*.

Consider the formula

$$A = p \wedge (\neg q \vee \neg p)$$

A is satisfiable if:

- $v_{\mathcal{I}}(p) = \text{true}$ and $v_{\mathcal{I}}(\neg q \vee \neg p) = \text{true}$
- $v_{\mathcal{I}}(\neg q \vee \neg p) = \text{true}$ iff $v_{\mathcal{I}}(\neg q) = \text{true}$ or $v_{\mathcal{I}}(\neg p) = \text{true}$

Semantic Tableaux (2)

Therefore, A is satisfiable if

- $v_{\mathcal{I}}(p) = \text{true}$ and $v_{\mathcal{I}}(\neg q) = \text{true}$, or
- $v_{\mathcal{I}}(p) = \text{true}$ and $v_{\mathcal{I}}(\neg p) = \text{true}$.

The second interpretation can never be satisfied as it contains a complementary pair of literals.

Therefore we can state that a set of literals can be satisfied **only if it does not contain** a complementary pair of literals.

$A = p \wedge (\neg q \vee \neg p)$ is satisfiable, but $A = p \wedge (\neg q \vee \neg p)$ is not valid.

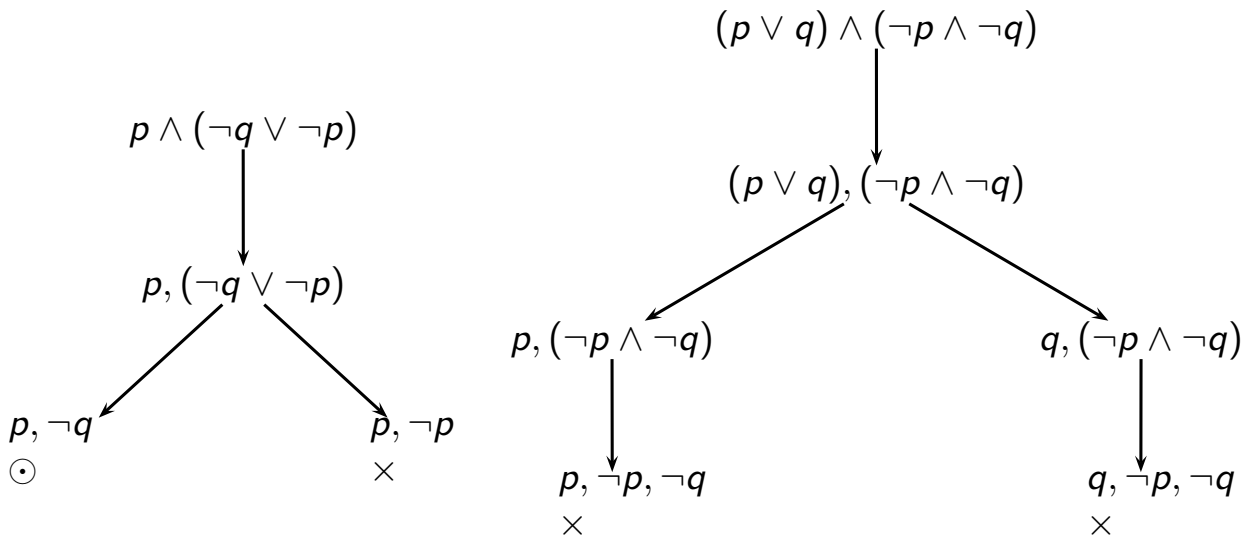
Semantic Tableaux (3)

We can use these ideas to build a technique for testing if a formula is satisfiable or valid.

- If a set of formulae contains a formula with a *conjunction*, we can replace the formula in the set of formulae by subformulae containing the terms of the conjunction.
- If a set of formulae contains a formula with a *disjunction*, we can replace the set of formulae by two sets of formulae, each containing one of the terms of the disjunction.
- A set of formulae is not satisfiable if it contains a complementary pair of formulae or a complementary pair of literals.

Semantic Tableaux (4)

Examples:



$\odot$ denotes an *open* set of literals This set is satisfiable..

$\times$ denotes an *closed* set of literals. This set is unsatisfiable.

Semantic Tableaux (5)

We can generalise this to other boolean operators.

1. If a set of formulae contains an α formula, replace α by α_1 and α_2 .
2. If a set of formulae contains a β formula, replace the set of formulae by two sets of formulae, one containing β_1 and the other containing β_2 .
3. Repeat step 1 or step 2 until each node only contains literals. A set of literals is closed if it contains a complementary pair of literals. Otherwise it is open.

α	α_1	α_2
$\neg\neg A_1$	A_1	
$A_1 \wedge A_2$	A_1	A_2
$\neg(A_1 \vee A_2)$	$\neg A_1$	$\neg A_2$
$\neg(A_1 \rightarrow A_2)$	A_1	$\neg A_2$
$\neg(A_1 \uparrow A_2)$	A_1	A_2
$A_1 \downarrow A_2$	$\neg A_1$	$\neg A_2$
$A_1 \leftrightarrow A_2$	$A_1 \rightarrow A_2$	$A_2 \rightarrow A_1$
$\neg(A_1 \oplus A_2)$	$A_1 \rightarrow A_2$	$A_2 \rightarrow A_1$

β	β_1	β_2
$\neg(B_1 \wedge B_2)$	$\neg B_1$	$\neg B_2$
$B_1 \vee B_2$	B_1	B_2
$B_1 \rightarrow B_2$	$\neg B_1$	B_2
$B_1 \uparrow B_2$	$\neg B_1$	$\neg B_2$
$\neg(B_1 \downarrow B_2)$	B_1	B_2
$\neg(B_1 \leftrightarrow B_2)$	$\neg(B_1 \rightarrow B_2)$	$\neg(B_2 \rightarrow B_1)$
$B_1 \oplus B_2$	$\neg(B_1 \rightarrow B_2)$	$\neg(B_2 \rightarrow B_1)$

Semantic Tableaux (6)

The construction of a semantic tableaux is guaranteed to terminate and each leaf of the tree (semantic tableaux) will be a set of literals.

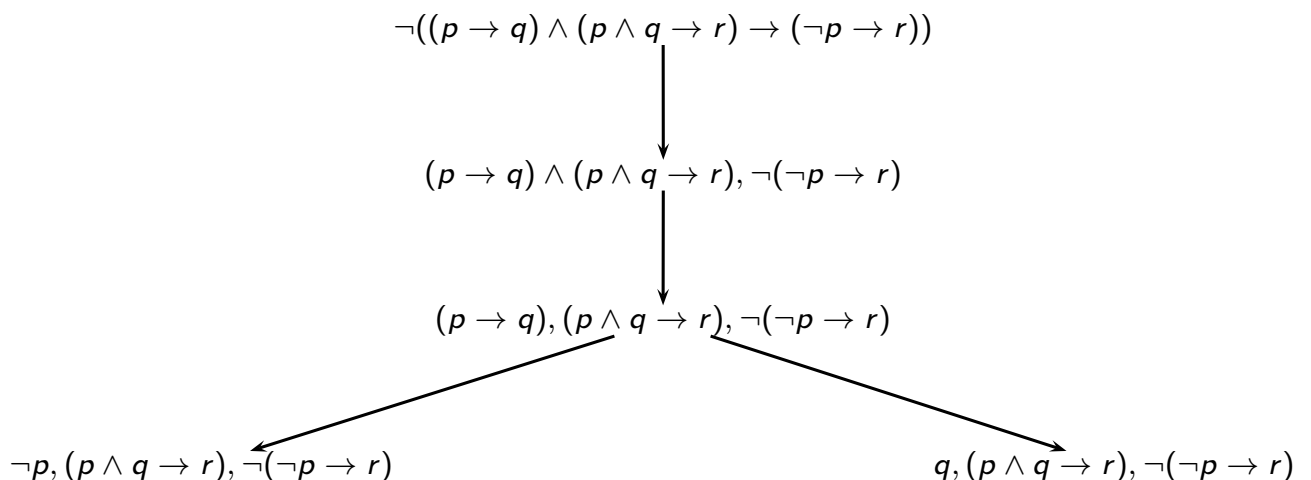
If all the leaves of the semantic tableaux are *closed*, then the semantic tableaux is closed. Otherwise the tableaux is *open*.

A is satisfiable **iff** its semantic tableaux is open.

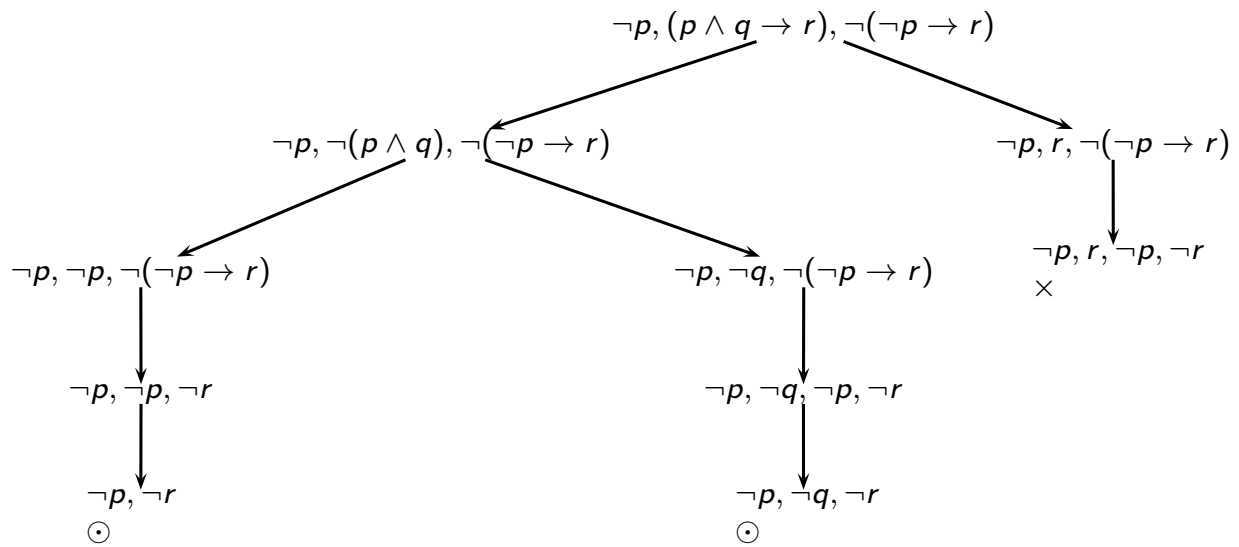
A is valid **iff** $\neg A$ is closed.

The semantic tableaux is a *refutation procedure*. The semantic tableaux for A is closed if A is unsatisfiable. To use semantic tableaux to prove A , i.e. show A is valid, we need to refute $\neg A$.

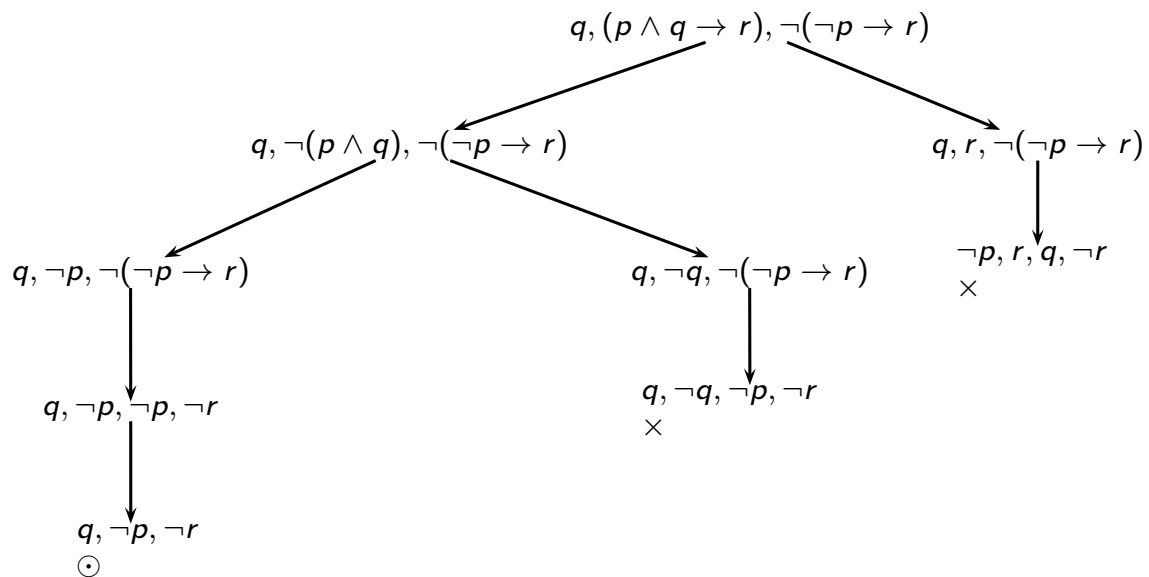
Semantic Tableaux (7)



Semantic Tableaux (8)



Semantic Tableaux (9)



Deductive Systems

A *deductive system* consists of:

- a set of formulae called *axioms*, and
- a set of *rules of inference*.

A *proof* in a deductive system is a sequence of formulae, $S = A_1, A_2, \dots, A_n$, such that each formula A_i is either an axiom or can be inferred from previous formulae in the sequence using a rule of inference.

The last formula in the sequence, A_n , is a *theorem*, and the sequence S is a *proof* of A_n .

If A_n is *provable*, denoted $\vdash A_n$, then A_n can be used as an axiom in subsequent proofs.

Gentzen System

In the *Gentzen System* $\mathcal{G}$ an axiom is a set of literals that contain a complementary pair of literals. A very important distinction between the sets of literals in the Gentzen System and the sets of literals in a semantic tableaux is:

- A set of literals in the Gentzen System are an *implicit disjunction*, i.e. $\{a, b, c\} \equiv a \vee b \vee c$.
- A set of literals in a semantic tableaux are an *implicit conjunction*, i.e. $\{a, b, c\} \equiv a \wedge b \wedge c$.

In the Gentzen System, there are two types of inference rules using the table on the following slide:

1. Let $\{\alpha_1, \alpha_2\} \subseteq U_1$ and let $U'_1 = U_1 - \{\alpha_1, \alpha_2\}$, then $U = U'_1 \cup \alpha$ can be inferred.
2. Let $\{\beta_1\} \subseteq U_1, \{\beta_2\} \subseteq U_2$ and let $U'_1 = U_1 - \{\beta_1\}$, $U'_2 = U_2 - \{\beta_2\}$ then $U = U'_1 \cup U'_2 \cup \{\beta\}$ can be inferred.

Gentzen System (2)

$$\frac{\vdash U'_1 \cup \{\alpha_1, \alpha_2\}}{\vdash U'_1 \cup \{\alpha\}}$$

$$\frac{\vdash U'_1 \cup \{\beta_1\} \quad \vdash U'_2 \cup \{\beta_2\}}{\vdash U'_1 \cup U'_2 \cup \{\beta\}}$$

α	α_1	α_2
$\neg\neg A$	A	
$\neg(A_1 \wedge A_2)$	$\neg A_1$	$\neg A_2$
$A_1 \vee A_2$	A_1	A_2
$A_1 \rightarrow A_2$	$\neg A_1$	A_2
$A_1 \uparrow A_2$	$\neg A_1$	$\neg A_2$
$\neg(A_1 \downarrow A_2)$	A_1	A_2
$\neg(A_1 \leftrightarrow A_2)$	$\neg(A_1 \rightarrow A_2)$	$\neg(A_2 \rightarrow A_1)$
$A_1 \oplus A_2$	$\neg(A_1 \rightarrow A_2)$	$\neg(A_2 \rightarrow A_1)$

β	β_1	β_2
$B_1 \wedge B_2$	B_1	B_2
$\neg(B_1 \vee B_2)$	$\neg B_1$	$\neg B_2$
$\neg(B_1 \rightarrow B_2)$	B_1	$\neg B_2$
$\neg(B_1 \uparrow B_2)$	B_1	B_2
$B_1 \downarrow B_2$	$\neg B_1$	$\neg B_2$
$B_1 \leftrightarrow B_2$	$B_1 \rightarrow B_2$	$B_2 \rightarrow B_1$
$\neg(B_1 \oplus B_2)$	$B_1 \rightarrow B_2$	$B_2 \rightarrow B_1$

$\frac{A}{B}$ is a *sequent* and means that if A is true, then B can be inferred.

We can drop the braces if we remove duplicates from a list of formulae.

Gentzen System (3)

Example: Prove $\vdash (p \vee q) \rightarrow (q \vee p)$ in $\mathcal{G}$.

Proof:

- | | | |
|----|--|-------------------------|
| 1. | $\vdash \neg p, q, p$ | axiom |
| 2. | $\vdash \neg q, q, p$ | axiom |
| 3. | $\vdash \neg(p \vee q), q, p$ | $\beta\vee, 1, 2$ |
| 4. | $\vdash \neg(p \vee q), (q \vee p)$ | $\alpha\vee, 3$ |
| 5. | $\vdash (p \vee q) \rightarrow (q \vee p)$ | $\alpha \rightarrow, 4$ |

However, knowing which axioms to start with, and which rules to apply and which order to apply them is difficult.

But working in the reverse direction solves all these problems.

- | | | |
|----|--|-------------------------|
| 1. | $\vdash (p \vee q) \rightarrow (q \vee p)$ | $\alpha \rightarrow, 2$ |
| 2. | $\vdash \neg(p \vee q), (q \vee p)$ | $\alpha\vee, 3$ |
| 3. | $\vdash \neg(p \vee q), q, p$ | $\beta\vee, 4, 5$ |
| 4. | $\vdash \neg q, q, p$ | axiom |
| 5. | $\vdash \neg p, q, p$ | axiom |

Gentzen System (4)

Example: Prove $\vdash (p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)$ in $\mathcal{G}$.

Proof:

- | | | |
|----|--|---------------------------|
| 1. | $\vdash p, q, \neg p$ | axiom |
| 2. | $\vdash \neg q, q, \neg p$ | axiom |
| 3. | $\vdash p, (\neg q \rightarrow \neg p)$ | $\alpha \rightarrow, 1$ |
| 4. | $\vdash \neg q, (\neg q \rightarrow \neg p)$ | $\alpha \rightarrow, 2$ |
| 5. | $\vdash \neg(p \rightarrow q), (\neg q \rightarrow \neg p)$ | $\beta \rightarrow, 3, 4$ |
| 6. | $\vdash (p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)$ | $\alpha \rightarrow, 5$ |

Again, in reverse:

- | | | |
|----|--|---------------------------|
| 1. | $\vdash (p \rightarrow q) \rightarrow (\neg q \rightarrow \neg p)$ | $\alpha \rightarrow, 2$ |
| 2. | $\vdash \neg(p \rightarrow q), (\neg q \rightarrow \neg p)$ | $\beta \rightarrow, 3, 4$ |
| 3. | $\vdash \neg q, (\neg q \rightarrow \neg p)$ | $\alpha \rightarrow, 5$ |
| 4. | $\vdash p, (\neg q \rightarrow \neg p)$ | $\alpha \rightarrow, 6$ |
| 5. | $\vdash \neg q, q, \neg p$ | axiom |
| 6. | $\vdash p, q, \neg p$ | axiom |

Gentzen System (5)

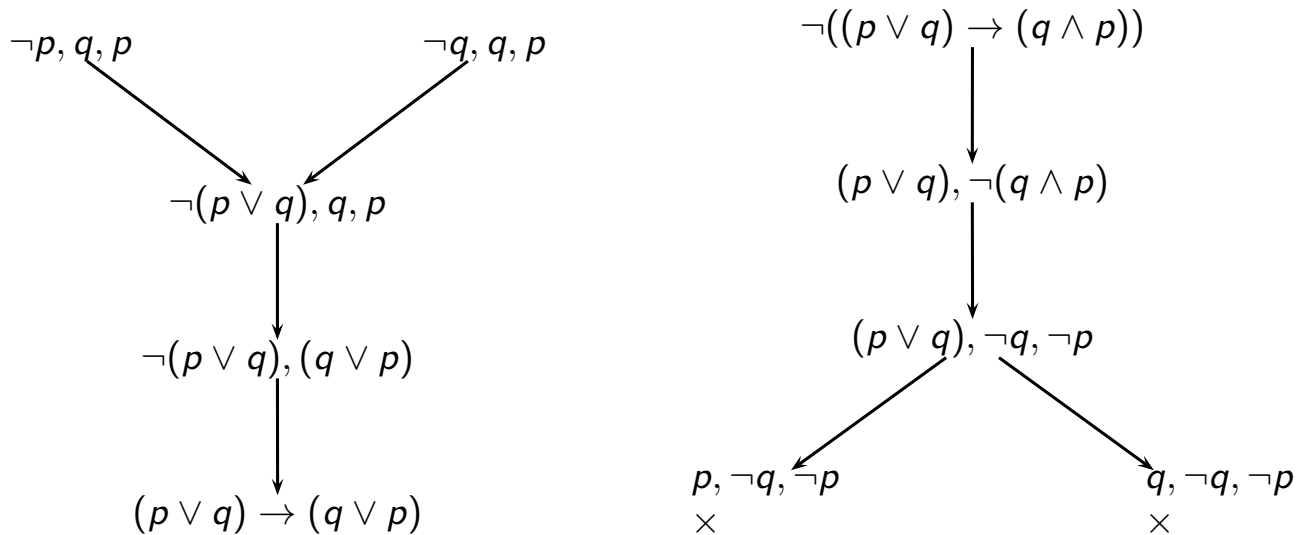
Example: Prove $\vdash (p \vee (q \wedge r)) \rightarrow (q \vee p) \wedge (p \vee r)$ in $\mathcal{G}$.

Proof (in the reverse direction):

- | | | |
|-----|---|-------------------------|
| 1. | $\vdash p \vee (q \wedge r) \rightarrow (p \vee q) \wedge (p \vee r)$ | $\alpha \rightarrow, 2$ |
| 2. | $\vdash \neg(p \vee (q \wedge r)), (p \vee q) \wedge (p \vee r)$ | $\beta \vee, 3, 8$ |
| 3. | $\vdash \neg p, (p \vee q) \wedge (p \vee r)$ | $\beta \wedge, 4, 6$ |
| 4. | $\vdash \neg p, (p \vee r)$ | $\alpha \vee, 5$ |
| 5. | $\vdash \neg p, p, r$ | axiom |
| 6. | $\vdash \neg p, (p \vee q)$ | $\alpha \vee, 7$ |
| 7. | $\vdash \neg p, p, q$ | axiom |
| 8. | $\vdash \neg(q \wedge r), (p \vee q) \wedge (p \vee r)$ | $\alpha \wedge, 9$ |
| 9. | $\vdash \neg q, \neg r, (p \vee q) \wedge (p \vee r)$ | $\beta \wedge, 10, 12$ |
| 10. | $\vdash \neg q, \neg r, (p \vee r)$ | $\alpha \vee, 11$ |
| 11. | $\vdash \neg q, \neg r, p, r$ | axiom |
| 12. | $\vdash \neg q, \neg r, (p \vee q)$ | $\alpha \vee, 13$ |
| 13. | $\vdash \neg q, \neg r, p, q$ | axiom |

Relationship between $\mathcal{G}$ and Semantic Tableaux

Because the list of literals in the Gentzen System is an implicit disjunction and the list of literals in a semantic tableaux are an implicit conjunction, then $\vdash A$ in $\mathcal{G}$ iff there is a closed semantic tableaux for $\neg A$.



“upside-down $\mathcal{G}$ proof as a tree”

CSC1018 David Sinclair ©2024

Conjunctive Normal Form

A formula is in *conjunctive normal form* (CNF) iff it is a conjunction of disjunctions of literals.

$$(a \vee \neg b \vee c) \wedge (\neg a \vee d) \wedge (b \vee c \vee d)$$

is in CNF, whereas

$$(a \vee (\neg b \wedge c)) \wedge (\neg a \vee d) \wedge (b \vee c \vee d)$$

is not in CNF (1st term is not a disjunction), and

$$(a \vee \neg b \vee c) \wedge \neg(a \vee d) \wedge (b \vee c \vee d)$$

is not in CNF (2nd term is negated).

Conjunctive Normal Form (2)

If a formula is not in CNF, it can be transformed into CNF by:

1. Use the following equivalences to remove all operators other than negations, conjunctions and disjunctions.

$$A \leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$$

$$A \oplus B \equiv \neg(A \rightarrow B) \vee \neg(B \rightarrow A)$$

$$A \rightarrow B \equiv \neg A \vee B$$

$$A \uparrow B \equiv \neg(A \wedge B)$$

$$A \downarrow B \equiv \neg(A \vee B)$$

2. Push negation inwards using De Morgan's laws.

$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

Conjunctive Normal Form (3)

3. Remove double negations.

$$\neg\neg A \equiv A$$

4. Use the distributivity law to remove conjunctions within disjunctions.

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

Example:

$$\begin{aligned} (\neg p \rightarrow \neg q) \rightarrow (p \rightarrow q) &\equiv \neg(\neg\neg p \vee \neg q) \vee (\neg p \vee q) \\ &\equiv (\neg\neg\neg p \wedge \neg\neg q) \vee (\neg p \vee q) \\ &\equiv (\neg p \wedge q) \vee (\neg p \vee q) \\ &\equiv (\neg p \vee \neg p \vee q) \wedge (q \vee \neg p \vee q) \end{aligned}$$

Clausal Form

The *clausal form* is another way of representing a formula in CNF.

- A *clause* is a set of literals.
- A clause is an implicit disjunction of literals.
- The empty set of literals is the *empty clause*, denoted $\square$.
- A formula in *clausal form* is a set of clauses.
- A formula is an implicit conjunction of its clauses.
- The formula that is the empty set of clauses is denoted $\emptyset$.

Example:

The CNF formula $(p \vee q \vee p) \wedge (\neg r \vee p) \wedge (\neg q \vee r)$ is logically equivalent to

$$\{\{p, q\}, \{\neg r, p\}, \{\neg q, r\}\}$$

also represented as

$$\{pq, \bar{r}p, \bar{q}r\}$$

Resolution

The empty clause, $\square$, is unsatisfiable. The empty set of clauses is valid.

Resolution is a *refutation procedure*. It determines if a formula A , represented in clausal form, is unsatisfiable. To prove A , i.e. show A is valid, then we need to show that $\neg A$ is unsatisfiable. If $\neg A$ is unsatisfiable, then A is valid.

Let C_1 and C_2 be clauses that contains a complementary literal, l , such that $l \in C_1$ and $l^c \in C_2$. C_1 and C_2 are *clashing clauses* and the *resolvent* of C_1 and C_2 is the clause:

$$\text{Res}(C_1, C_2) = (C_1 - \{l\}) \cup (C_2 - \{l^c\})$$

A clause is *trivial* if it contains a pair of complementary literals.

Resolution (2)

Resolution procedure: Given a set of clauses, S .

- Let $S_0 = S$
- Repeat the following steps to generate S_{i+1} from S_i until the procedure terminate.
 - Choose a pair of clashing clauses $C_1, C_2 \subseteq S_i$ that have **not been chosen before**.
 - Let $C = \text{Res}(C_1, C_2)$
 - If C is trivial then $S_{i+1} = S_i$ else $S_{i+1} = S_i \cup \{C\}$.
 - Terminate if:
 - $C = \square$. S is unsatisfiable.
 - All pairs of clashing clauses have been resolved. S is satisfiable.

Resolution (3)

Example:

Using Resolution prove $(p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))$ is valid.

If $A = (p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))$, then A is valid if $\neg A$ is unsatisfiable.

$$\begin{aligned}
 \neg A &\equiv \neg((p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))) \\
 &\equiv \neg((\neg p \vee (\neg q \vee r)) \rightarrow (\neg(\neg p \vee q) \vee (\neg p \vee r))) \\
 &\equiv \neg(\neg(\neg p \vee (\neg q \vee r)) \vee (\neg(\neg p \vee q) \vee (\neg p \vee r))) \\
 &\equiv \neg\neg(\neg p \vee (\neg q \vee r)) \wedge (\neg(\neg(\neg p \vee q) \vee (\neg p \vee r))) \\
 &\equiv (\neg p \vee (\neg q \vee r)) \wedge (\neg\neg(\neg p \vee q) \wedge \neg(\neg p \vee r)) \\
 &\equiv (\neg p \vee \neg q \vee r) \wedge (\neg p \vee q) \wedge (\neg\neg p \wedge \neg r) \\
 &\equiv (\neg p \vee \neg q \vee r) \wedge (\neg p \vee q) \wedge p \wedge \neg r \\
 \neg A &\equiv \{\overline{p}qr, \overline{p}q, p, \overline{r}\}
 \end{aligned}$$

Resolution (4)

$$S_0 = \{\overline{pqr}, \overline{pq}, p, \overline{r}\}$$

$\overline{pqr}$ and $\overline{r}$ clash on r , hence:

$$S_1 = \{\overline{pq}, p, \overline{pq}\}$$

$\overline{pq}$ and $\overline{pq}$ clash on q , hence:

$$S_2 = \{p, \overline{p}\}$$

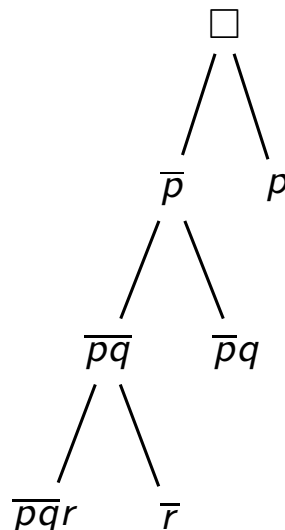
p and $\overline{p}$ clash on p , hence:

$$S_3 = \{\square\}$$

Hence $\neg A$ is unsatisfiable and A is valid.

Resolution (5)

Another way of visualising the resolution process is as a tree:



Resolution as a Proof Technique

Suppose we want to prove a *Prolog-style* statement:

$A :- B, C, D.$

This is equivalent to:

$$\begin{aligned} & A \leftarrow B \wedge C \wedge D \\ \equiv & \neg(B \wedge C \wedge D \wedge \neg A) \quad \text{since } X \rightarrow Y \equiv \neg(X \wedge \neg Y) \end{aligned}$$

As Resolution is a *refutation* procedure, in order to prove $A \leftarrow B \wedge C \wedge D$, we need to refute $B \wedge C \wedge D \wedge \neg A$.

Example using Resolution

Either Bob attended the meeting or Bob was not invited. If the boss wanted Bob at the meeting, then Bob was invited. Bob did not attend the meeting. If the boss did not want Bob there, and then the boss did not invite him, then he is going to be fired. Using *Resolution* **prove** Bob is going to be fired.

Step 1: Define propositions

- Let A be true if Bob attended the meeting.
- Let I be true if Bob was invited.
- Let W be true if the boss wanted Bob at the meeting.
- Let F be true if Bob is going to be fired.

Example using Resolution (2)

Step 2: Convert text into premises.

Text	Proposition	CNF
Either Bob attended the meeting or Bob was not invited.	$A \vee \neg I$	$A\bar{I}$
If the boss wanted Bob at the meeting, then Bob was invited.	$W \rightarrow I$	$\overline{WI}$
Bob did not attend the meeting.	$\neg A$	$\bar{A}$
If the boss did not want Bob there, and then the boss did not invite him, then he is going to be fired.	$(\neg W \wedge \neg I) \rightarrow F$	WIF

Step 3: Negate conclusion

Text	Negation of conclusion	CNF
Bob is going to be fired.	$\neg F$	$\bar{F}$

Example using Resolution (3)

Step 4: Resolution.

$$\{A\bar{I}, \overline{WI}, \bar{A}, WIF, \bar{F}\}$$

Resolve WIF and $\bar{F}$

$$\{A\bar{I}, \overline{WI}, \bar{A}, WI\}$$

Resolve WI and $\overline{WI}$

$$\{A\bar{I}, I, \bar{A}\}$$

Resolve $A\bar{I}$ and I

$$\{A, \bar{A}\}$$

Resolve A and $\bar{A}$

$$\{\square\}$$

Another Example using Resolution

If Jane loves Pat, then Jane loves John. If it is Monday, Jane loves Pat or John. Prove using *Resolution* that if it is Monday, then Jane loves John.

Step 1: Define propositions

- Let p be true if Jane loves Pat.
- Let j be true if Jane loves John.
- Let m be true if it is Monday.

Step 2: Convert text into premises.

Text	Proposition	CNF
If Jane loves Pat, then Jane loves John.	$p \rightarrow j$	$\bar{p}j$
If it is Monday, Jane loves Pat or John.	$m \rightarrow (p \vee j)$	$\bar{m}pj$

CSC1018 David Sinclair ©2024

Another Example using Resolution (2)

Step 3: Negate conclusion

Text	Negation of conclusion	CNF
If it is Monday, then Jane loves John.	$\neg(m \rightarrow j)$	$m \bar{j}$

Note: 2 separate negated goals since

$$\neg(m \rightarrow j) \equiv \neg(\neg m \vee j) \equiv m \wedge \neg j.$$

Step 4: Resolution.

$$\{\bar{p}j, \bar{m}pj, m, \bar{j}\}$$

Resolve $\bar{m}pj$ and m

$$\{\bar{p}j, pj, \bar{j}\}$$

Another Example using Resolution (3)

Resolve $\bar{p}j$ and pj

$$\{j, \bar{j}\}$$

Resolve j and $\bar{j}$

$$\{\square\}$$